

Java Operators

Operators are special symbols used to perform operations on variables and values. They are essential for performing calculations, comparing values, making decisions, and manipulating data.

Java operators are classified into the following categories:

1. Arithmetic Operators
2. Unary Operators
3. Assignment Operators
4. Relational Operators
5. Logical Operators
6. Bitwise Operators
7. Shift Operators
8. Ternary Operator
9. instanceof Operator

Arithmetic Operators

Arithmetic operators are used to perform mathematical calculations.

Operator	Name	Example	Result
+	Addition	10 + 5	15
-	Subtraction	10 - 5	5
*	Multiplication	10 * 5	50
/	Division	10 / 5	2
%	Modulus (Remainder)	10 % 3	1

```
public class Main {  
    public static void main(String[] args) {  
  
        int a = 10, b = 3;  
  
        System.out.println("Addition: " + (a + b));  
        System.out.println("Subtraction: " + (a - b));  
        System.out.println("Multiplication: " + (a * b));  
        System.out.println("Division: " + (a / b));  
        System.out.println("Modulus: " + (a % b));  
    }  
}
```

Addition: 13
Subtraction: 7
Multiplication: 30
Division: 3
Modulus: 1

Unary Operators

Unary operators operate on a single operand.

Operator	Name	Description
+	Unary Plus	Indicates a positive value
-	Unary Minus	Negates a value
++	Increment	Increases value by 1
--	Decrement	Decreases value by 1
!	Logical NOT	Reverses a boolean value
~	Bitwise Complement	Reverses all bits

```

public class Main {
    public static void main(String[] args) {

        int a = 10;

        System.out.println(+a); // 10
        System.out.println(-a); // -10

        System.out.println(++a); // 11
        System.out.println(a++); // 11
        System.out.println(a); // 12

        System.out.println(--a); // 11
        System.out.println(a--); // 11
        System.out.println(a); // 10

        boolean flag = true;
        System.out.println(!flag); // false
    }
}

```

Pre-increment vs Post-increment

++a Increment first, then use the value

a++ Use the current value first, then increment

```

int a = 5;
System.out.println(++a); // 6
int b = 5;
System.out.println(b++); // 5
System.out.println(b); // 6

```

Assignment Operators

Assignment operators are used to assign values to variables.

Operator	Example	Equivalent Expression	Answer
=	a = 10	Assigns 10 to a	a = 10
+=	a += 5	a = a + 5	a = 15
-=	a -= 5	a = a - 5	a = 5
*=	a *= 5	a = a * 5	a = 50
/=	a /= 5	a = a / 5	a = 2
%=	a %= 5	a = a % 5	a = 0
&=	a &= 5	a = a & 5	a = 0
=	a = 5	a = a 5	a = 15
^=	a ^= 5	a = a ^ 5	a = 15
<<=	a <<= 2	a = a << 2	a = 40
>>=	a >>= 2	a = a >> 2	a = 2
>>>=	a >>>= 2	a = a >>> 2	a = 2

```

public class Main {
    public static void main(String[] args) {

        int a = 10;
    }
}

```

```

    a += 5;
    System.out.println(a); // 15

    a -= 3;
    System.out.println(a); // 12

    a *= 2;
    System.out.println(a); // 24

    a /= 4;
    System.out.println(a); // 6

    a %= 4;
    System.out.println(a); // 2
}
}

```

Relational Operators

Relational operators are used to compare two values. They return a boolean result: true or false.

Operator	Name	Example	Result
==	Equal to	10 == 10	true
!=	Not equal to	10 != 5	true
>	Greater than	10 > 5	true
<	Less than	10 < 5	false
>=	Greater than or equal to	10 >= 10	true
<=	Less than or equal to	5 <= 10	true

```

public class Main {
    public static void main(String[] args) {

        int a = 10, b = 20;

        System.out.println(a == b); // false
        System.out.println(a != b); // true
        System.out.println(a > b); // false
        System.out.println(a < b); // true
        System.out.println(a >= b); // false
        System.out.println(a <= b); // true
    }
}

```

Logical Operators

Logical operators are used to combine or reverse boolean expressions.

Operator	Name	Description
&&	Logical AND	True if both conditions are true
	Logical OR	True if at least one condition is true
!	Logical NOT	Reverses the boolean result

```

public class Main {
    public static void main(String[] args) {

```

```
int age = 20;
boolean hasID = true;

System.out.println(age >= 18 && hasID);
System.out.println(age >= 18 || hasID);
System.out.println(!(age >= 18));
}
}
true
true
false
```

Bitwise Operators

Bitwise operators perform operations on individual bits of integer values.

a = 5 → 0101

b = 3 → 0011

Operator	Name	Example	Result
&	Bitwise AND	5 & 3	1
	Bitwise OR	5 3	7
^	Bitwise XOR	5 ^ 3	6
~	Bitwise Complement	~5	-6

```
public class Main {
    public static void main(String[] args) {

        int a = 5, b = 3;

        System.out.println(a & b);
        System.out.println(a | b);
        System.out.println(a ^ b);
        System.out.println(~a);
    }
}
1
7
6
-6
```

Shift Operators

Shift operators move the bits of a number to the left or right.

Operator	Name	Description
<<	Left Shift	Shifts bits to the left
>>	Signed Right Shift	Shifts bits right, preserving the sign
>>>	Unsigned Right Shift	Shifts bits right, filling with zeros

Operator	Name	Example	Binary Before	Binary After	Result
<<	Left Shift	8 << 1	00001000	00010000	16
<<	Left Shift	8 << 2	00001000	00100000	32
>>	Signed Right Shift	8 >> 1	00001000	00000100	4
>>	Signed	8 >> 2	00001000	00000010	2

	Right Shift				
>>	Signed Right Shift	-8 >> 1	11111000	11111100	-4
>>>	Unsigned Right Shift	8 >>> 1	00001000	00000100	4
>>>	Unsigned Right Shift	-8 >>> 1	11111111...11111000	01111111...11111100	214748364

```

public class Main {
    public static void main(String[] args) {

        int a = 8;

        System.out.println(a << 1);
        System.out.println(a << 2);
        System.out.println(a >> 1);
        System.out.println(a >> 2);
        System.out.println(a >>> 1);
    }
}
16
32
4
2
4

```

Ternary Operator

The ternary operator is a shorthand alternative to a simple if-else statement

```

public class Main {
    public static void main(String[] args) {

        int a = 10, b = 20;

        int max = (a > b) ? a : b;

        System.out.println("Maximum: " + max);
    }
}
Maximum: 20

```

instanceof Operator

The instanceof operator checks whether an object is an instance of a particular class or its subclass, or implements a particular interface. It returns a boolean value.

```

class Animal {
}

class Dog extends Animal {
}

public class Main {

```

```
public static void main(String[] args) {  
  
    Dog d = new Dog();  
  
    System.out.println(d instanceof Dog);  
    System.out.println(d instanceof Animal);  
    System.out.println(d instanceof Object);  
}  
}  
true  
true  
true
```

- d instanceof Dog → true because d refers to a Dog object.
- d instanceof Animal → true because Dog inherits Animal.
- d instanceof Object → true because all Java classes inherit from Object, directly or indirectly.